SoloLLM Documentation

Mukhtasar Taaruf (Introduction)

SoloLLM ek complete local AI assistant application hai jo aapke computer par bilkul privately chalta hai. Yeh ChatGPT jaisi functionality deta hai lekin aapka data kabhi bhi internet par nahi jata.

Kya Hai SoloLLM?

SoloLLM ek self-hosted AI chatbot hai jisme yeh khasiyat hain:

1. **100% Local Execution** - Tamaam AI processing aapke computer par hoti hai
2. **Privacy First** - Koi data bahar nahi jata
3. **Ollama Integration** - Popular LLM models automatically download hote hain
4. **RAG System** - Apne documents upload karke unse sawal jawab karein
5. **Self-Training** - Apni conversations se model ko train karein (LoRA fine-tuning)
6. **Agent Mode** - AI tools ke saath complex tasks complete karein
7. **Knowledge Graph** - Documents se entities aur relationships extract karein
8. **Web Search** - Internet search with AI summarization

Project Structure

```
SololLm/ ■■■ backend/ # Python FastAPI backend server ■ ■■■ api/ # REST API endpoints ■ ■■■ core/ # Core business logic ■ ■■■ rag/ # RAG (Retrieval Augmented Generation) system ■ ■■■ memory/ # Knowledge Graph ■ ■■■ storage/ # Database layer ■ ■■■ main.py # Application entry point ■ ■■■ requirements.txt # Python dependencies ■ ■■■ frontend/ # Next.js React frontend ■ ■■■ src/ ■ ■ ■■■ app/ # Main app pages ■ ■ ■■■ components/ # UI components ■ ■ ■■■ lib/ # API client ■ ■ ■■■ types/ # TypeScript interfaces ■ ■■■ package.json # Node.js dependencies ■ ■■■ data/ # Runtime data (created on first run) ■ ■■■ db/ # SQLite database ■ ■■■ documents/ # Uploaded documents ■ ■■■ models/ # Downloaded Ollama models ■ ■■■ ollama/ # Embedded Ollama binary ■ ■■■ training/ # Training outputs ■ ■■■ docs/ # Yeh documentation folder ■■■ backend/ # Backend ki detailed documentation ■■■ frontend/ # Frontend ki detailed documentation ■■■ technologies/ # Istemal hui technologies ki detail
```

Quick Start Guide

System Requirements

- **Windows 10/11** ya Linux/macOS
- **RAM**: Minimum 8GB (16GB recommended)
- **Storage**: 10GB+ free space (models ke liye)
- **GPU (Optional)**: NVIDIA GPU with 4GB+ VRAM for faster inference

Installation Steps

Python Environment Setup

```
cd backend python -m venv .venv .venv\Scripts\activate # Windows # ya: source .venv/bin/activate # Linux/macOS pip install -r requirements.txt
```

Frontend Setup

```
cd frontend npm install
```

Run the Application

Terminal 1 (Backend):

```
cd backend python -m uvicorn main:app --reload --port 8000
```

Terminal 2 (Frontend):

```
cd frontend npm run dev
```

Open Browser

```
http://localhost:3000
```

Pehli Baar Run Karte Waqt

Jab aap pehli baar application chalayein:

1. **Ollama Auto-Download** - Backend automatically Ollama binary download karega (~100MB)
2. **Setup Wizard** - Frontend mein ek wizard dikhega jo hardware detect karega
3. **Model Selection** - Aap compatible models mein se choose kar sakte hain
4. **Model Pull** - Selected model download hoga (size model par depend karta hai)

Documentation Index

Backend Documentation

- Backend Overview - Backend architecture ka overview
- API Endpoints - Complete API reference
- Core Modules - Core logic ki detail
- RAG System - Document processing aur retrieval
- Database - Database schema aur functions
- Training - Self-training system ki detail

Frontend Documentation

- Frontend Overview - Frontend architecture
- Components - UI components ki detail
- API Client - Backend se communication
- Types - TypeScript interfaces

Technologies Documentation

- Technologies Overview - Tamaam technologies ka overview
- Python Libraries - Python packages ki detail
- Node.js Packages - Frontend dependencies
- AI/ML Technologies - LoRA, RAG, Embeddings waghaira
- Downloads & Setup - Background mein kya download hota hai

Features Overview

1. Chat Interface

- Real-time streaming responses (SSE)
- Multiple conversations support
- Thread-based context isolation
- Message continuation (truncated responses ke liye)

2. RAG (Retrieval Augmented Generation)

- PDF, TXT, MD, HTML, DOCX support
- Automatic chunking aur embedding
- Vector search (FAISS)
- Keyword search (BM25-style)
- Citations in responses

3. Self-Training (QLoRA)

- Conversation history se training data generate
- Document-based training
- GPU/CPU auto-detection
- Ollama mein model registration

4. Agent Mode

- Calculator, Code Runner, File Reader/Writer
- Web Search aur Web Scraper
- RAG Search tool
- Knowledge Graph queries
- Persistent agent memory

5. Knowledge Graph

- Entity extraction from documents

- Relationship mapping
- Graph visualization
- Community detection

6. Dashboard & Export

- Performance metrics
- Request/Response analytics
- Conversation export/import
- Settings backup

Support & Contribution

Agar koi sawal ya masla ho:

1. GitHub Issues mein report karein
2. Documentation check karein
3. Logs dekhein: `backend/` folder mein console output

Version Info

- **Current Version**: 0.1.0
- **Ollama Version**: 0.6.2
- **Python**: 3.11+
- **Node.js**: 18+
- **Next.js**: 16.1.6
- **FastAPI**: 0.115.6